

Experiment 5

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory:

Solidity is a programming language used for developing smart contracts on blockchain platforms such as Ethereum. Smart contracts contain programmable logic that can be deployed and executed on the blockchain.

1. **Primitive Data Types, Variables, Functions – pure, view**

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int** → unsigned and signed integers of different sizes, such as `uint256` and `int128`.
- **bool** → represents logical values such as `true` or `false`.
- **address** → stores a 20-byte Ethereum account or contract address.
- **bytes / string** → used to store binary or textual data.

2. Variables in Solidity can be:

- **State variables** → stored permanently on the blockchain.
- **Local variables** → temporary variables created during function execution.
- **Global variables** → predefined variables such as `msg.sender`, `msg.value`, and `block.timestamp`.

3. Functions execute the logic of a smart contract. Two important function types are:

- **pure** → cannot read or modify blockchain state and works only with inputs and internal computations.
- **view** → can read state variables but cannot modify them.

4. This classification helps optimize gas usage and maintain function integrity.

5. **Inputs and Outputs to Functions**

Functions in Solidity can accept input arguments and return one or more output values.

- **Inputs** allow users or other contracts to pass data to a function.
- **Outputs** allow the function to return the result of a computation.
- Solidity also supports **named return variables**, which improve readability and debugging.

6. For example, a function can accept an amount in Ether and return whether a transfer was successful.

7. Visibility, Modifiers and Constructors

a. Function Visibility

Function visibility determines who can access a function:

- **public** → available both inside and outside the contract.
- **private** → accessible only within the same contract.
- **internal** → accessible within the contract and its child contracts.
- **external** → can be called by external accounts or other contracts.

8. b. Modifiers

Modifiers are reusable code blocks that change the behavior of functions. They are commonly used for access control, such as restricting sensitive functions to the contract owner using an `onlyOwner` modifier.

c. Constructors

Constructors are special functions executed only once during contract deployment. They are generally used to initialize important values, such as setting the deploying account as the owner.

9. Control Flow: if-else, loops

Solidity provides control-flow statements similar to traditional programming languages.

- **if-else** → used for conditional decision-making, such as checking whether a balance is sufficient before transferring funds.
 - **Loops** → `for`, `while`, and `do-while` loops are used for repeated execution.
10. Loops should be used carefully because excessive iterations can increase gas consumption and make contract execution expensive.

11. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays** → used to store ordered lists of elements. They can be fixed-size or dynamic.
- **Mappings** → key-value pairs that provide quick lookups. For example, `mapping(address => uint)` can store balances. Mappings do not support iteration.
- **Structs** → allow related properties to be grouped into a single data type. Example:
`struct Player { string name; uint score; }`

- **Enums** → define a set of predefined constants and improve code readability.

Example:

```
enum Status { Pending, Active, Closed }
```

12. Data Locations

Solidity provides three primary data locations:

- **storage** → data is stored permanently on the blockchain. State variables are examples of storage data.
- **memory** → temporary data storage that exists only while a function is executing.
- **calldata** → non-modifiable and non-persistent data location used for external function parameters. It is generally more gas-efficient than memory.

13. Understanding data locations is important because they directly affect gas costs and performance.

14. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei** → Ether is the main currency of Ethereum. All values are measured in Wei, where:

$$1 \text{ Ether} = 10^{18} \text{ Wei}$$

- **Gas and Gas Price** → every transaction consumes gas, which represents the computational effort required. Gas price determines how much Ether is paid per unit of gas.
- **Sending Transactions** → transactions are used to transfer Ether or interact with smart contracts. Functions such as `transfer()`, `send()`, and `call()` can be used for transferring Ether.

15. Every transaction requires gas, so efficient smart-contract design is important.

Output:

Tutorial No. 1:

The screenshot displays the Remix IDE interface. On the left, the 'Environment' panel shows the 'Remix VM' and 'Account 1' with a balance of 100.999 ETH. The main editor shows a Solidity contract named 'Counter' with three functions: 'get()', 'inc()', and 'dec()'. The 'Deploy & Run Transactions' panel on the right shows the 'get()' function being called, resulting in a return value of 0. The 'Transactions recorder' at the bottom shows the transaction details, including the sender's address, the gas used, and the transaction hash.

REMIX2.5.5

Learneth Solidity Beginner Course

LinkSSee PlansGet AI Credits

DEPLOY & RUN TRANSACTIONS

Environment

ForkReset

Remix VM

Osaka

Account 1

0x5B3...eddC4

100.999 ETH

Low level interaction

+

Inc

Value

0

wei

Gas limit

auto

0

Transact

Balance

0 ETH

Transactions recorder 2

Save

Create a DApp with your contracts

Start now

4

contract Counter {

5

uint public count;

6

7

// Function to get the current count

8

function get() public view returns (uint) { 2453 gas

9

return count;

10

}

11

12

// Function to increment count by 1

13

function inc() public { infinite gas

14

count += 1;

15

}

16

17

// Function to decrement count by 1

18

function dec() public { infinite gas

19

count -= 1;

20

}

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

1001

1002

1003

1004

1005

1006

1007

1008

1009

1010

1011

1012

1013

1014

<

REMIX 2.5.5

Learneth Solidity Beginner Course

Link S See Plans Get AI Credits

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Remix VM Osaka

Account 1 100.999 ETH
0x5B3...eddC4

Low level interaction +

dec

Value 0 wei

Gas limit auto 0

Transact

Balance 0 ETH

Transactions recorder 3 Save

Create a DApp with your contracts Start now

```
4 contract Counter {
5   uint public count;
6
7   // Function to get the current count
8   function get() public view returns (uint) { 2453 gas
9     return count;
10  }
11
12  // Function to increment count by 1
13  function inc() public { infinite gas
14    count += 1;
15  }
16
17  // Function to decrement count by 1
18  function dec() public { infinite gas
19    count -= 1;
20  }
21 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

call [call] from: 0x5B380a6a701c568545dcfc803fc8875f56beddC4 to: Counter.count() data: 0x066...61abd Debug

[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xc29...dEc11 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0x741...5526d Debug

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

REMIX 2.5.5

Learneth Solidity Beginner Course

Link S See Plans Get AI Credits

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Remix VM Osaka

Account 1 100.999 ETH
0x5B3...eddC4

Functions Select a function to interact with...

Low level interaction +

count

0uint256: 0

Call

Balance 0 ETH

Create a DApp with your contracts Start now

```
4 contract Counter {
5   uint public count;
6
7   // Function to get the current count
8   function get() public view returns (uint) { 2453 gas
9     return count;
10  }
11
12  // Function to increment count by 1
13  function inc() public { infinite gas
14    count += 1;
15  }
16
17  // Function to decrement count by 1
18  function dec() public { infinite gas
19    count -= 1;
20  }
21 }
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: Counter.dec() 0xc29...dEc11 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0x741...5526d Debug

call [call] from: 0x5B380a6a701c568545dcfc803fc8875f56beddC4 to: Counter.count() data: 0x066...61abd Debug

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial No. 2:

REMITX 2.5.5 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

2. Basic Syntax | 2 / 19

WE DEFINE THE **visibility** OF THE VARIABLE, WHICH SPECIFIES FROM WHERE YOU CAN ACCESS IT. IN THIS CASE, IT'S A **public** VARIABLE THAT YOU CAN ACCESS FROM INSIDE AND OUTSIDE THE CONTRACT.

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the *creator* of the Solidity by Example contracts.

Watch a video tutorial on Basic Syntax.

★ **Assignment**

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer **Show answer**

Next

Well done! No errors.

Explain contract | AI copilot

0 ☐ Listen on all transactions | Filter with transaction hash or address

• ethers.js

Type the library name to see available commands.

✓ [vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21

status 1 Transaction mined and execution completed

transaction hash 0xc1b97f42bb0975608fedc98c6280ba2c214c3a96cb3d934f1b17b0e63b5b21

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial No. 3:

REMITX 2.5.5 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

3. Primitive Data Types | 3 / 19

ALL THESE DATA TYPES HAVE DEFAULT VALUES, AS SHOWN IN THE CONTRACT (LINE 29).

You can learn more about these data types as well as *Fixed Point Numbers*, *Byte Arrays*, *Strings*, and more in the *Solidity documentation*.

Later in the course, we will look at data structures like **Mappings**, **Arrays**, **Enums** and **Structs**.

Watch a video tutorial on Primitive Data Types.

★ **Assignment**

1. Create a new variable **newAddr** that is a **public** **address** and give it a value that is not the same as the available variable **addr**.
2. Create a **public** variable called **neg** that is a negative number, decide upon the type.
3. Create a new variable, **newU** that has the smallest **uint** size type and the smallest **uint** value and is **public**.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer **Show answer**

Next

Well done! No errors.

Explain contract | AI copilot

0 ☐ Listen on all transactions | Filter with transaction hash or address

• ethers.js

Type the library name to see available commands.

✓ [vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xc1b...b5b21

status 1 Transaction mined and execution completed

transaction hash 0xc1b97f42bb0975608fedc98c6280ba2c214c3a96cb3d934f1b17b0e63b5b21

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial No. 4:

REMX 2.5.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

4. Variables 4 / 19

function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

RemixAI Copilot (enabled)

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public {
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Here are some global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender; // address of the caller
17        blockNumber = block.number;
18    }
19 }
```

Tutorial No. 5:

REMX 2.5.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.1 Functions - Reading and Writing to a State Variable 5 / 19

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a [video tutorial on Functions](#).

★ Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer Show answer

Next

Well done! No errors.

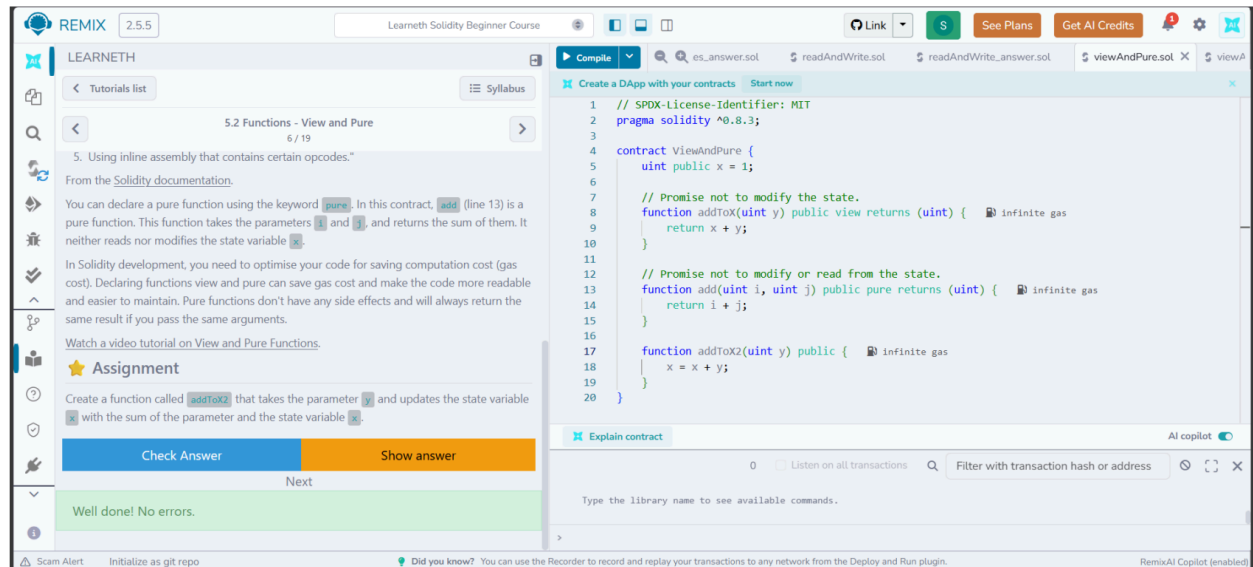
Scam Alert Initialize as git repo

Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin.

RemixAI Copilot (enabled)

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8
9     // You need to send a transaction to write to a state variable.
10    function set(uint _num) public {
11        num = _num;
12    }
13
14    // You can read from a state variable without sending a transaction.
15    function get() public view returns (uint) {
16        return num;
17    }
18
19    function get_b() public view returns (bool) {
20        return b;
21    }
22 }
```

Tutorial No. 6:



REMITX 2.5.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.2 Functions - View and Pure 6 / 19

5. Using inline assembly that contains certain opcodes.*
From the Solidity documentation.
You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.
In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.
Watch a video tutorial on View and Pure Functions.

★ Assignment

Create a function called `addToX` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

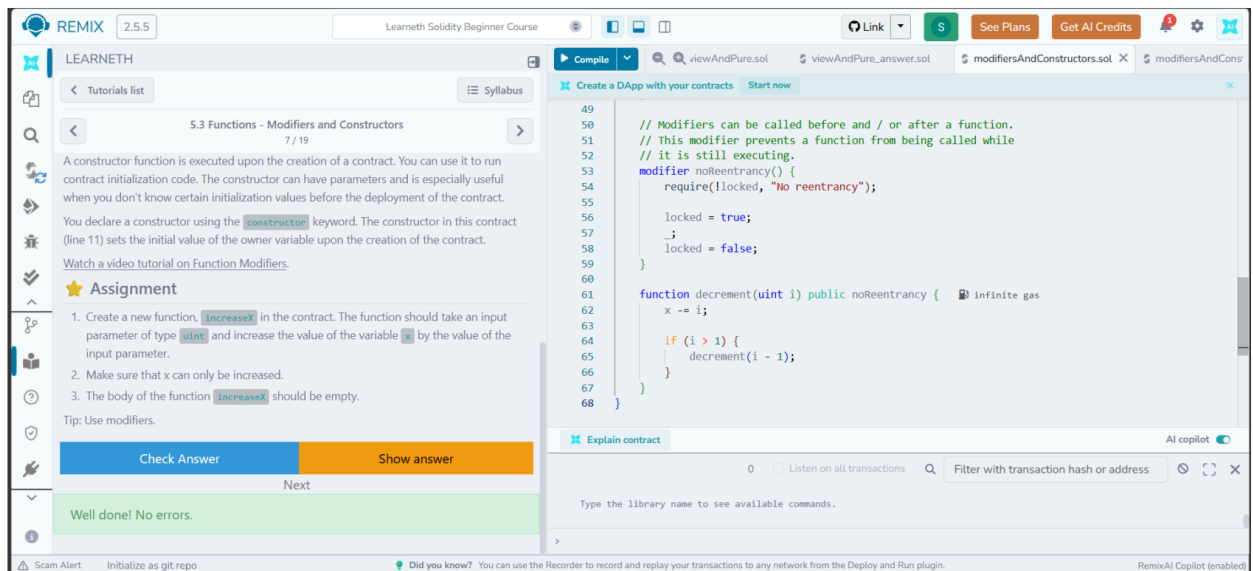
```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) { infinite gas
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) { infinite gas
14        return i + j;
15    }
16
17    function addToX2(uint y) public { infinite gas
18        x = x + y;
19    }
20 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

Tutorial No. 7:



REMITX 2.5.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.3 Functions - Modifiers and Constructors 7 / 19

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.
You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.
Watch a video tutorial on Function Modifiers.

★ Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

```
49
50 // Modifiers can be called before and / or after a function.
51 // This modifier prevents a function from being called while
52 // it is still executing.
53 modifier noReentrancy() {
54     require(!locked, "No reentrancy");
55
56     locked = true;
57     _;
58     locked = false;
59 }
60
61 function decrement(uint i) public noReentrancy { infinite gas
62     x -= i;
63
64     if (i > 1) {
65         decrement(i - 1);
66     }
67 }
68 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

Tutorial No. 8:

REMITX 2.5.5 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

5.4 Functions - Inputs and Outputs | 8 / 19

There are a few restrictions and best practices for the input and output parameters of contract functions.

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ **Assignment**

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer **Show answer**

Next

Well done! No errors.

Scam Alert Initialize as git repo **Did you know?** You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. **RemixAI Copilot (enabled)**

```
72
73 // Can use array for output
74 uint[] public arr;
75
76 function arrayOutput() public view returns (uint[] memory) {
77     return arr;
78 }
79
80
81 function returnTwo()
82     public
83     pure
84     returns (
85         int i,
86         bool b
87     )
88 {
89     i = -2;
90     b = true;
91 }
```

Explain contract | AI copilot

0 ☐ Listen on all transactions

Type the library name to see available commands.

Tutorial No. 9:

REMITX 2.5.5 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

6. Visibility | 9 / 19

- Can be called from other contracts or transactions
- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

★ **Assignment**

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer **Show answer**

Next

Well done! No errors.

Scam Alert Initialize as git repo **Did you know?** You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. **RemixAI Copilot (enabled)**

```
51 // State variables cannot be external so this code won't compile.
52 // string external externalVar = "my external variable";
53 }
54
55 contract Child is Base {
56     // Inherited contracts do not have access to private functions
57     // and state variables.
58     // function testPrivateFunc() public pure returns (string memory) {
59     //     return privateFunc();
60     // }
61
62     // Internal function call be called inside child contracts.
63     function testInternalFunc() public pure override returns (string memory) {
64         return internalFunc();
65     }
66
67     function testInternalVar() public view returns (string memory, string memory) {
68         return (internalVar, publicVar);
69     }
70 }
```

Explain contract | AI copilot

0 ☐ Listen on all transactions

Type the library name to see available commands.

Tutorial No. 10:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

7.1 Control Flow - If/Else 10 / 19

the other conditions are met.

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the `If/Else` statement.

★ **Assignment**

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract IfElse {
5     function foo(uint x) public pure returns (uint) {
6         if (x < 10) {
7             return 0;
8         } else if (x < 20) {
9             return 1;
10        } else {
11            return 2;
12        }
13    }
14
15    function ternary(uint _x) public pure returns (uint) {
16        // if (_x < 10) {
17        //     return 1;
18        // }
19        // return 2;
20    }
21    // shorthand way to write if / else statement
```

Console:

0 ☐ Listen on all transactions

Type the library name to see available commands.

RemixAI Copilot (enabled)

Tutorial No. 11:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

7.2 Control Flow - Loops 11 / 19

continue

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second `if` statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the `break` statement (line 14) will cause the `for` loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

★ **Assignment**

1. Create a public `uint` state variable called `count` in the `Loop` contract.
2. At the end of the `for` loop, increment the `count` variable by 1.
3. Try to get the `count` variable to be equal to 9, but make sure you don't edit the `break` statement.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
8     for (uint i = 0; i < 10; i++) {
9         if (i == 5) {
10             // Skip to next iteration with continue
11             continue;
12         }
13         if (i == 5) {
14             // Exit loop with break
15             break;
16         }
17         count++;
18     }
19
20     // while loop
21     uint j;
22     while (j < 10) {
23         j++;
24     }
25 }
26
27
```

Console:

0 ☐ Listen on all transactions

Type the library name to see available commands.

RemixAI Copilot (enabled)

Tutorial No. 12:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

8.1 Data Structures - Arrays 12 / 19

(line 42). When we remove an element with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length
Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

★ **Assignment**

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
54 // Move the last element into the place to delete
55 arr[index] = arr[arr.length - 1];
56 // Remove the last element
57 arr.pop();
58 }
59
60 function test() public {
61     arr.push(1);
62     arr.push(2);
63     arr.push(3);
64     arr.push(4);
65     // [1, 2, 3, 4]
66
67     remove(1);
68     // [1, 4, 3]
69
70     remove(2);
71     // [1, 4]
72 }
73 }
```

Tutorial No. 13:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

8.2 Data Structures - Mappings 13 / 19

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values
We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

★ **Assignment**

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
38 bool _boo
39 } public {
40     nested[_addr1][_i] = _boo;
41 }
42
43 function remove(address _addr1, uint _i) public {
44     delete nested[_addr1][_i];
45 }
46 }
```

Tutorial No. 14:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

8.3 Data Structures - Structs 14 / 19

mapping inside curly braces (line 19).
Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs
To access a member of a struct we can use the dot operator (line 33).

Updating structs
To update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

★ **Assignment**
Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
34 }
35
36 // update text
37 function update(uint _index, string memory _text) public {
38     Todo storage todo = todos[_index];
39     todo.text = _text;
40 }
41
42 // update completed
43 function toggleCompleted(uint _index) public {
44     Todo storage todo = todos[_index];
45     todo.completed = !todo.completed;
46 }
47
48 function remove(uint _index) public {
49     delete todos[_index];
50 }
51 }
```

Tutorial No. 15:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

8.4 Data Structures - Enums 15 / 19

Updating an enum value
We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value
We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ **Assignment**

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Solidity Code:

```
28
29 function getSize() public view returns (Size) {
30     return sizes;
31 }
32
33 // Update status by passing uint into input
34 function set(Status _status) public {
35     status = _status;
36 }
37
38 // You can update to a specific enum like this
39 function cancel() public {
40     status = Status.Canceled;
41 }
42
43 // delete resets the enum to its first value, 0
44 function reset() public {
45     delete status;
46 }
47 }
```

Tutorial No. 16:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

9. Data Locations 16 / 19

not affect the values stored in the mapping `myStructs` (line 10).
As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ **Assignment**

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myNewStruct2` with the data location `memory` inside the `function f` and assign it the value of `myNewStruct`. Change the value of the `myNewStruct2` member `foo` to 1.
3. Create a new struct `myNewStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myNewStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myNewStruct2`, and `myNewStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer **Show answer**

Next

Well done! No errors.

Explain contract

```
36 // You can return memory variables
37 function g(uint[] memory _arr) public returns (uint[] memory) {
38     // do something with memory array
39     _arr[0] = 1;
40 }
41
42 function h(uint[] calldata _arr) external {
43     // do something with calldata array
44     _arr[0] = 1;
45 }
46
47 }
```

Tutorial No. 17:

LEARNETH 2.5.5 Learneth Solidity Beginner Course

10.1 Transactions - Ether and Wei 17 / 19

Wei is the smallest subunit of *Ether*, named after the cryptographer *Wei Dai*. *Ether* numbers without a suffix are treated as `wei` (line 7).

`1 gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`1 ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ **Assignment**

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer **Show answer**

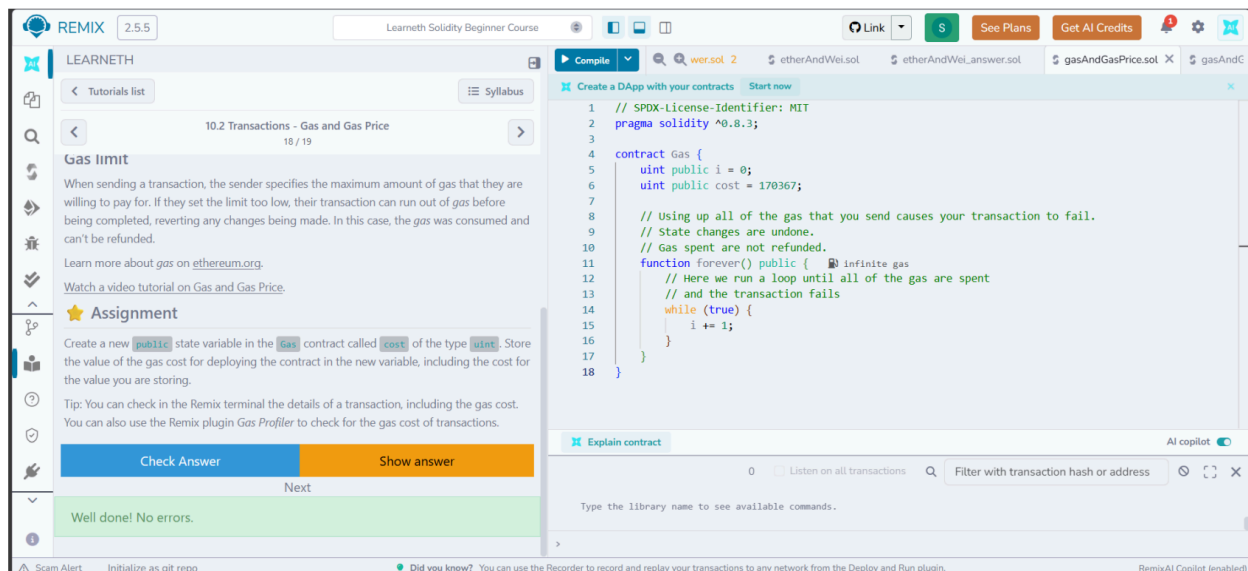
Next

Well done! No errors.

Explain contract

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneGwei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneGwei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
```

Tutorial No. 18:



LEARNETH 2.5.5 Learneth Solidity Beginner Course

Tutorials list 10.2 Transactions - Gas and Gas Price 18 / 19

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.
Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin `Gas Profiler` to check for the gas cost of transactions.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public { infinite gas
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

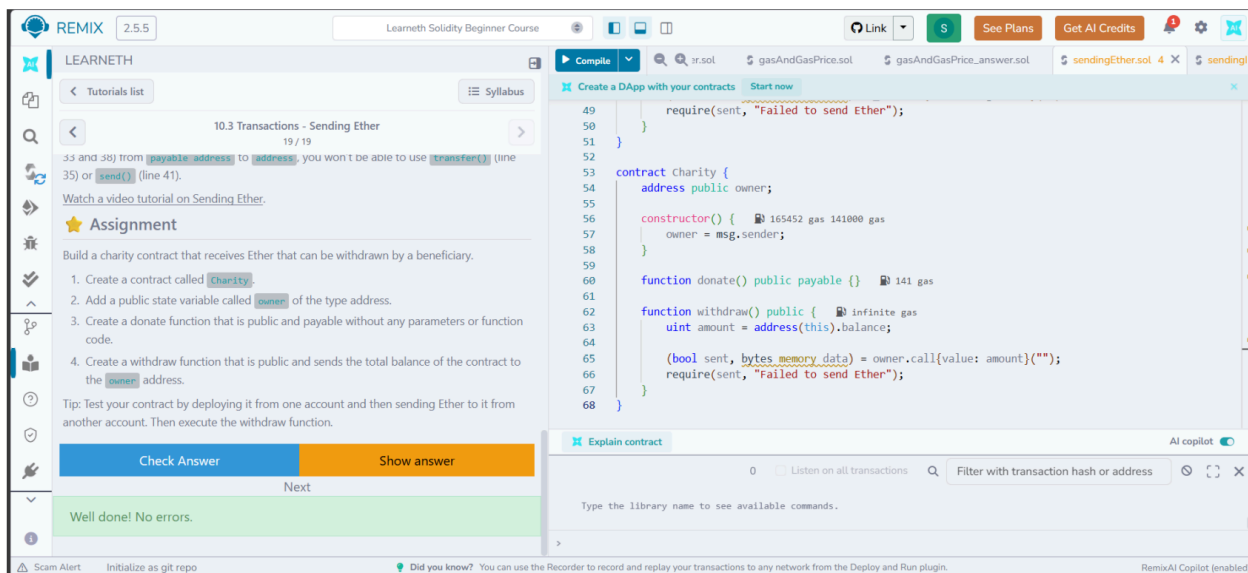
Explain contract AI copilot

0 ☐ Listen on all transactions

Type the library name to see available commands.

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Tutorial No. 19:



LEARNETH 2.5.5 Learneth Solidity Beginner Course

Tutorials list 10.3 Transactions - Sending Ether 19 / 19

33 and 38) from `payable_address` to `address`, you won't be able to use `transfer()` (line 35) or `send()` (line 41).

Watch a video tutorial on Sending Ether.

Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer **Show answer**

Next

Well done! No errors.

```
49 }
50 require(sent, "Failed to send Ether");
51 }
52
53 contract Charity {
54     address public owner;
55
56     constructor() { 165452 gas 141000 gas
57         owner = msg.sender;
58     }
59
60     function donate() public payable { 141 gas
61
62     function withdraw() public { infinite gas
63         uint amount = address(this).balance;
64
65         (bool sent, bytes memory data) = owner.call{value: amount}("");
66         require(sent, "Failed to send Ether");
67     }
68 }
```

Explain contract AI copilot

0 ☐ Listen on all transactions

Type the library name to see available commands.

Scam Alert Initialize as git repo Did you know? You can use the Recorder to record and replay your transactions to any network from the Deploy and Run plugin. RemixAI Copilot (enabled)

Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.